

Skill 3

Interactive Shell Loop and Keyboard Input Handling

Aim

To create an interactive command loop, display a shell prompt, read user input, handle exit conditions, design the control flow and implement keyboard input handling including backspace, Enter and multi-character commands.

Requirements

- Ubuntu Linux
- GCC
- C programming environment
- Terminal

Step-by-Step Implementation

Step 1: Create the Source File

```
nano src/shell_loop.c
```

Step 2: Display the Prompt

The shell displays a prompt to indicate that it is ready to receive a command.

Example:

```
printf("myshell> ");  
fflush(stdout);
```

Step 3: Read User Input

Input can be read using:

```
fgets(buffer, sizeof(buffer), stdin);
```

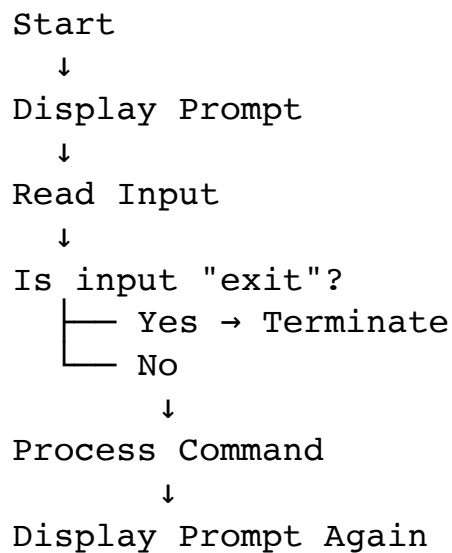
Step 4: Remove the Newline

```
buffer[strcspn(buffer, "\n")] = '\0';
```

Step 5: Handle Exit Condition

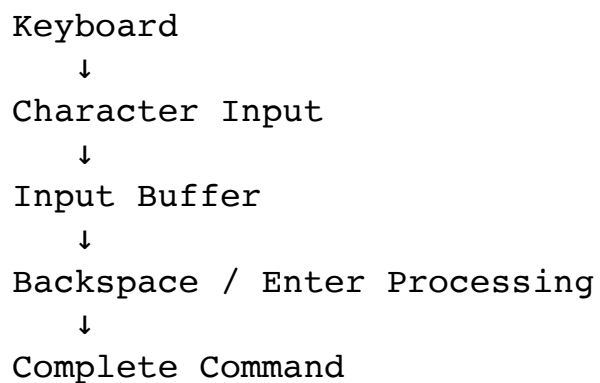
```
if (strcmp(buffer, "exit") == 0) {  
    break;  
}
```

Step 6: Implement the Main Loop



Step 7: Handle Keyboard Input

The input buffer stores characters entered by the user.



Step 8: Handle Backspace

When a backspace character is received, the most recently entered character is removed from the input buffer.

Step 9: Handle Enter

The Enter key indicates that the user has finished entering the command and the buffer can be processed.

Step 10: Test Multi-Character Commands

Example:

```
myshell> hello world
myshell> test command
myshell> exit
```

Observation

The interactive shell repeatedly displayed the prompt and accepted user input. The Enter key was used to complete commands, while backspace allowed characters to be removed. Multi-character commands were stored in the input buffer and processed after completion.

End Result

An interactive command loop with keyboard input handling was successfully designed and tested.